Plan de Proyecto

📅 12 de Septiembre: Presentación de Asignatura y Proyecto

Esta jornada marcó el inicio oficial del ciclo de desarrollo de aplicaciones web (DAW) para el proyecto de fin de curso (TFG). La sesión se dividió en tres bloques fundamentales: metodología de trabajo, identificación del problema y viabilidad técnica.

1. Marco Metodológico y Estándares de Calidad

Durante la presentación de la asignatura por parte del tutor, **Francisco Mera Calderón**, se establecieron los criterios de evaluación y los hitos obligatorios que regirían el curso.

- **Enfoque Profesional**: Se hizo hincapié en que el TFG no debe ser solo un ejercicio académico, sino una solución tecnológica real con potencial de mercado.
- **Gestión del Tiempo**: Se introdujo el concepto de desarrollo iterativo, donde cada fase de la documentación (Manuales, Bases de Datos, Despliegue) debe estar alineada con el avance del código.

2. Análisis de Problemas en el Sector Servicios

Se realizó una fase de investigación preliminar para detectar ineficiencias en negocios de proximidad. Tras un análisis de campo, se puso el foco en la **Tintorería "Jerez"**, extrayendo las siguientes conclusiones:

- **Cuello de Botella Logístico**: Se observó que el local acumula una cantidad crítica de prendas listas que no son reclamadas por los clientes en los plazos previstos.
- **Coste de Oportunidad**: Este espacio ocupado impide la entrada de nuevos pedidos de gran volumen (edredones, alfombras), limitando la facturación potencial del negocio.
- **Deficiencia en la Comunicación**: Se detectó que la comunicación actual es reactiva (esperar a que el cliente venga) o manual (llamadas telefónicas individuales), lo cual es ineficiente y propenso al error humano.

3. Conceptualización de "CleanManager"

Como respuesta a estas necesidades, se procedió a la definición de la idea de proyecto bajo el nombre de **CleanManager**.

- **Visión del Producto**: Una plataforma web *full-stack* capaz de gestionar integralmente el negocio, desde el registro de la prenda hasta la liberación automática del espacio.
- **Propuesta de Valor**: La automatización de notificaciones vía WhatsApp/Email como herramienta de "desbloqueo" logístico, avisando al cliente no solo cuando la prenda está lista, sino mediante recordatorios de escalada a los 7 y 15 días.
- **Impacto Esperado**: Reducción drástica del stock inmovilizado y mejora de la satisfacción del cliente al recibir información en tiempo real.

4. Formalización del Equipo de Trabajo

Se concluyó la jornada formalizando la propuesta con **Jose Antonio González Mejias** como el alumno responsable del diseño, desarrollo y despliegue del sistema, bajo la tutorización directa del departamento de informática.

📅 Cronograma Detallado: Fase de Inicio

Fecha	Hito Principal	Actividades Realizadas	Entregable / Resultado
12-Sep	**Presentación y Lanzamiento del Proyecto**	• Sesión de presentación de la asignatura por parte de la jefatura de estudios. • Análisis de las directrices y criterios de evaluación del TFG. • Sesión de *brainstorming* para la identificación de problemas en negocios locales	**Documento de Definición de Objetivos**: Establecimiento de la meta principal (reducción de ropa acumulada) y elección formal de la empresa beneficiaria.

📅 19 de Septiembre: Creación de Imagen Corporativa de la Empresa

Tras la definición de los objetivos del proyecto en la sesión anterior, esta jornada se dedicó íntegramente a la construcción de la **identidad visual corporativa**. El diseño de una imagen sólida no es solo una cuestión estética; es una herramienta estratégica para transmitir confianza a los clientes de la tintorería y profesionalidad en el software de gestión.

1. Conceptualización del Naming: "CleanManager"

El primer paso fue la elección del nombre comercial. Se optó por **"CleanManager"** debido a que comunica directamente la propuesta de valor del software:

- **"Clean"**: Referencia directa al sector de la limpieza y tintorería, aportando claridad sobre el nicho de mercado.
- **"Manager"**: Atributo de gestión, orden y control, reforzando la idea de que la aplicación soluciona el caos logístico del local.
- **Slogan**: Se barajaron opciones que enfatizaran la rapidez y el orden, pilares fundamentales para resolver el problema de la acumulación de ropa.

2. Psicología del Color y Tipografía

Se realizó un estudio de colorimetría para seleccionar una paleta que evocara los valores de higiene, tecnología y eficiencia:

- **Azul Cian/Primario**: Elegido por su asociación psicológica con la limpieza, el agua y la confianza. Es un color estándar en el sector de las tintorerías que genera calma en el usuario.
- **Blanco y Gris Claro**: Utilizados para las superficies de la interfaz, buscando transmitir amplitud y orden, contrastando con el problema real del local: la congestión de espacio.
- **Tipografía**: Se seleccionaron fuentes de la familia *Sans Serif* (modernas y sin remates) para garantizar la legibilidad en pantallas de diversos tamaños, facilitando el trabajo rápido del operario en el local.

3. Diseño del Isotipo y Logotipo

Se procedió a la creación de una representación gráfica que sintetizara la funcionalidad del sistema:

- **Simbología**: El diseño incorporó elementos abstractos que sugieren una prenda de ropa (como una percha o una silueta de camisa) integrada con elementos tecnológicos (como ondas de señal o un check de verificación).
- **Versatilidad**: Se diseñaron versiones del logo en positivo, negativo y escala de grises para asegurar que la imagen corporativa sea funcional tanto en el encabezado de la web como en los tickets impresos o notificaciones digitales.

4. Aplicación de la Marca en la Experiencia de Usuario (UX)

La creación de la imagen corporativa no se limitó a un logo, sino que definió las pautas de diseño para todo el aplicativo:

- **Coherencia Visual**: La paleta de colores definida este día servirá de base para la configuración de **Tailwind CSS**, asegurando que todos los botones de "Estado de Pedido" y alertas sigan una línea visual coherente.
- **Percepción de Profesionalidad**: Una interfaz con una identidad bien definida aumenta la "eficacia percibida" del sistema de notificaciones, haciendo que el cliente final confíe más en los recordatorios que recibe en su dispositivo.

5. Documentación y Activos Digitales

Al finalizar la jornada, se generó un pequeño **Manual de Estilo** básico que incluye:

- Códigos hexadecimales de los colores corporativos.
- Logotipos en alta resolución (formatos SVG y PNG).
- Guía de uso de iconos para representar los estados de las prendas (Recibido, En proceso, Listo), lo que facilitará la fase de desarrollo de interfaces prevista para octubre.

📅 26 de Septiembre: Contrato de Servicios y Recogida de Necesidades

1. Elaboración del Contrato de Prestación de Servicios TIC

Para profesionalizar la relación entre el desarrollador (**Jose Antonio Gonzalez Mejias**) y la entidad beneficiaria, se redactó un contrato formal de

servicios tecnológicos. Este documento garantiza la seriedad del TFG y define las responsabilidades de ambas partes:

- **Objeto del Contrato**: El desarrollo, implementación y mantenimiento de una plataforma web denominada **CleanManager** para la gestión integral del establecimiento.
- **Propiedad Intelectual y Cesión de Uso**: Se estableció que, aunque la autoría intelectual pertenece al alumno como parte de su formación en DAW, la empresa tiene derecho de uso de la herramienta para su operativa diaria.
- **Confidencialidad de Datos (RGPD)**: Dado que la aplicación manejará datos sensibles (nombres, teléfonos y correos electrónicos de clientes), se incluyó una cláusula estricta de protección de datos según la normativa vigente.
- **Compromiso de Colaboración**: La empresa se comprometió a facilitar el acceso a la información necesaria sobre sus procesos de lavado y planchado para que el software refleje fielmente la realidad del negocio.

2. Recogida de Necesidades y Diagnóstico de Problemas

Tras la firma del contrato, se realizó una entrevista de campo con el responsable para identificar los "puntos de dolor" del negocio:

A. El Problema de la Inmovilización de Espacio Físico

- Se detectó que una cantidad significativa de prendas limpias permanecen en las perchas mucho más tiempo del debido.

- Este stock inmovilizado genera una saturación en el local, dificultando la entrada de pedidos voluminosos como edredones o alfombras.
- La falta de espacio físico impide una distribución eficiente de la maquinaria y el flujo de trabajo de los operarios.

B. Deficiencias en el Sistema de Comunicación Actual

- El método de aviso actual se basa en llamadas telefónicas manuales que consumen mucho tiempo del personal y a menudo no son contestadas.
- Se identificó que muchos clientes simplemente olvidan que tienen ropa pendiente, y la falta de un recordatorio automático perpetúa el problema.
- Existe una carencia total de historial digital, lo que impide saber con exactitud cuántos días lleva una prenda ocupando sitio en el almacén.

3. Definición de la Solución Propuesta (CleanManager)

Basándose en las necesidades recogidas, se acordó que la aplicación debe atacar estos problemas mediante la automatización:

- **Digitalización del Inventario**: Sustituir los procesos manuales por un CRUD (Crear, Leer, Actualizar, Borrar) que gestione clientes, pedidos y cada prenda de forma individualizada.
- **Automatización de Notificaciones**: Implementar un sistema que envíe avisos automáticos cuando el pedido cambie de estado a "Listo".
- **Sistema de Escalada de Avisos**: Se definieron plazos críticos para enviar recordatorios automáticos si la prenda no es recogida (ej. a los

7 y 15 días), incluyendo advertencias sobre posibles cargos por almacenaje para incentivar la recogida rápida.

- **Dashboard de Control**: El administrador necesita visualizar KPIs (Indicadores Clave de Rendimiento) para conocer en todo momento el valor total de la "ropa abandonada" y la eficacia de las notificaciones.

4. Portal de Autoservicio para el Cliente

Como necesidad adicional para mejorar la atención, se planteó que los clientes puedan registrarse para consultar el estado de sus pedidos (En proceso, Listo, Pendiente de recogida) de forma autónoma, reduciendo así la carga de consultas telefónicas en el local.

📅 3 de Octubre: Definición de Requisitos Funcionales y No Funcionales

Tras la recogida de información inicial, el objetivo de este día fue establecer el "qué" debe hacer la aplicación (**Requisitos Funcionales**) y "cómo" debe comportarse a nivel de rendimiento, seguridad y usabilidad (**Requisitos No Funcionales**).

1. Definición Detallada de Requisitos Funcionales (RF)

Los requisitos funcionales definen los servicios específicos que el sistema debe proporcionar para resolver los problemas de logística, espacio y comunicación.

- **RF1: Gestión de Usuarios y Autenticación**: El sistema debe implementar un módulo de login y registro robusto tanto para el administrador de la tintorería como para los clientes.
- **RF2: Módulo Backoffice (CRUD)**: El administrador debe poder crear, leer, actualizar y eliminar (CRUD) registros de clientes, pedidos y prendas de forma centralizada.
- **RF3: Trazabilidad y Gestión de Estados**: La aplicación debe permitir el cambio de estado de cada pedido a través de un flujo definido: "Recibido", "En proceso", "Listo" y "Recogido".
- **RF4: Automatización de Notificaciones por Evento**: El sistema debe disparar automáticamente una notificación (email o SMS) al cliente en el momento exacto en que su pedido pase al estado "Listo".
- **RF5: Sistema de Recordatorios Escalados**: Para combatir la inmovilización de espacio, el sistema debe enviar avisos automáticos adicionales a los 7 días de estancia y un aviso crítico a los 15 días, informando sobre posibles cargos por almacenaje.
- **RF6: Dashboard de Control y KPIs**: La interfaz del administrador debe incluir un panel con indicadores clave de rendimiento, como el número de pedidos pendientes de recogida y el valor total de la ropa "abandonada".
- **RF7: Portal de Autoservicio para Clientes**: Los clientes deben disponer de un área privada donde, previo login, puedan consultar el historial de sus pedidos y el estado actual de sus prendas.

2. Definición de Requisitos No Funcionales (RNF)

Estos requisitos garantizan la calidad técnica, la seguridad y la viabilidad operativa del proyecto a largo plazo.

- **RNF1: Seguridad y Autorización**: Implementar un sistema de autenticación y autorización robusto para proteger la integridad de los datos de los clientes y de la empresa.
- **RNF2: Diseño Adaptativo (Responsive)**: Aunque el proyecto excluye una app móvil nativa, la web debe ser totalmente responsive para funcionar correctamente en tablets y smartphones.
- **RNF3: Usabilidad Profesional**: La interfaz debe ser intuitiva y profesional, facilitando el uso por parte del personal de la tintorería sin necesidad de formación extensa.
- **RNF4: Persistencia de Datos**: Utilización de una base de datos relacional para garantizar la consistencia de la información y la integridad de las relaciones entre pedidos y clientes.

3. Presentación y Validación con la Empresa

Una vez redactados estos puntos, se realizó una reunión con la dirección de la tintorería "Jerez" para presentar el documento de requisitos y obtener su validación oficial.

- **Validación del Flujo de Notificaciones**: La empresa aprobó los plazos de 7 y 15 días para los recordatorios, considerándolos esenciales para liberar el valioso espacio físico del local.
- **Aprobación del Dashboard**: Se enfatizó la importancia de visualizar la eficacia de las notificaciones enviadas para medir la mejora en la rotación de prendas.

- **Acuerdo de Alcance (MVP)**: Se ratificó que el Producto Mínimo Viable se centraría en la gestión web y notificaciones automáticas, posponiendo integraciones de hardware o pagos online para futuras actualizaciones.

📅 10 de Octubre: Desarrollo de las Interfaces Gráficas

El diseño de las interfaces se centró en dos frentes principales: un **Backoffice** eficiente para el personal de la tintorería y un **Portal de Autoservicio** accesible para los clientes.

1. Conceptualización del Diseño UI/UX

Se aplicaron principios de diseño centrado en el usuario para garantizar que la aplicación fuera una herramienta de ayuda y no una carga administrativa:

- **Diseño Limpio y Profesional**: Se utilizó una estética minimalista basada en la paleta de colores corporativa (azules y blancos) para transmitir orden e higiene.
- **Enfoque en la Usabilidad (Responsive)**: Dado que el personal utilizará el sistema en el local, se diseñaron interfaces adaptables para tablets y smartphones, facilitando la movilidad dentro de la tintorería.
- **Jerarquía Visual**: Se priorizaron los elementos de acción rápida, como el cambio de estado de las prendas y la visualización de alertas urgentes.

2. Diseño del Backoffice para el Administrador

Esta interfaz es el núcleo operativo de **CleanManager**. Se diseñaron los siguientes componentes:

- **Dashboard Principal**: Un panel visual que incluye gráficos simples (mediante librerías como Chart.js) para mostrar los **KPIs** definidos: pedidos pendientes de recogida y eficacia de las notificaciones.
- **Módulo de Gestión de Pedidos (CRUD)**: Una tabla dinámica con filtros de búsqueda rápida por nombre o teléfono, permitiendo al operario crear o editar pedidos y clientes de forma ágil.
- **Selector de Estados**: Se implementó un sistema de botones o "steppers" visuales para cambiar el estado de la ropa (Recibido, En proceso, Listo, Recogido) de manera intuitiva.
- **Configurador de Notificaciones**: Una pantalla para que el administrador pueda personalizar los textos de los mensajes y los plazos de las alertas de recogida.

3. Diseño del Portal del Cliente

Se buscó la máxima sencillez para fomentar el uso por parte de usuarios finales:

- **Vista de Estado de Pedidos**: Una interfaz clara donde el cliente puede ver una lista de sus prendas y una barra de progreso que indica si están en lavado, listas para recoger o ya entregadas.
- **Historial de Servicios**: Un área para consultar pedidos pasados, permitiendo al cliente tener un registro de su consumo en la tintorería.

- **Perfil de Usuario**: Pantalla simplificada para la actualización de datos de contacto (email/teléfono), asegurando que las notificaciones lleguen correctamente.

4. Flujos de Navegación y Prototipado

Se definieron los caminos que seguirá el usuario dentro de la aplicación:

- **Flujo de Entrada**: Desde el login seguro hasta el dashboard de control.
- **Flujo de Notificación**: Se diseñó visualmente el mensaje que el cliente recibirá vía Email, asegurando que incluya los datos clave: número de pedido, importe y recordatorio de recogida.

5. Herramientas y Metodología

Para este proceso se utilizaron herramientas de prototipado modernas que permiten visualizar el resultado final antes de escribir el código:

- **Diseño de Componentes**: Se planificó el uso de componentes reutilizables (botones, tarjetas, modales) para que el desarrollo posterior con **React** y **Tailwind CSS** fuera más estructurado y eficiente.
- **Validación de Diseño**: Los prototipos fueron revisados bajo la óptica del "uso real" en la tintorería, asegurando que el flujo de trabajo diseñado ahorre tiempo y resuelva la congestión del local.

📅 17 de Octubre: Desarrollo de la Estructura de la Base de Datos

El diseño de la base de datos se realizó bajo un modelo relacional, utilizando **PostgreSQL** (a través de Supabase) debido a su robustez y capacidad para gestionar relaciones complejas entre clientes, pedidos y prendas. El objetivo principal fue crear un esquema normalizado que evitara la redundancia y permitiera consultas rápidas para el dashboard de control.

1. Identificación de Entidades y Atributos

Se procedió a desglosar el sistema en sus componentes de datos esenciales, definiendo los campos necesarios para cada uno:

- **Entidad Clientes**: Diseñada para almacenar la información de contacto necesaria para el portal de autoservicio y la automatización de notificaciones.
    - *Atributos*: ID único (UUID), nombre completo, correo electrónico, número de teléfono (campo crítico para futuros mensajes), contraseña cifrada y fecha de registro.
- **Entidad Pedidos**: El núcleo de la gestión logística, encargada de centralizar la trazabilidad de cada servicio.
    - *Atributos*: ID de pedido, ID del cliente (clave foránea), estado actual (Recibido, En proceso, Listo, Recogido), fecha de entrada, fecha prevista de entrega y coste total.

- **Entidad Prendas**: Una tabla de detalle que permite desglosar cada pedido en artículos individuales, facilitando un control preciso del inventario.
    - *Atributos*: ID de prenda, ID del pedido (clave foránea), tipo de prenda (traje, edredón, camisa), servicio solicitado (lavado, planchado, seco) y observaciones técnicas.
- **Entidad Notificaciones**: Creada para auditar la comunicación y controlar los plazos de recogida.
    - *Atributos*: ID de notificación, ID del cliente, tipo de aviso (Aviso de "Listo", Recordatorio 7 días, Aviso crítico 15 días) y marca de tiempo del envío.

2. Diseño de la Lógica de Estados e Integridad

Se definieron las reglas de negocio a nivel de base de datos para asegurar que el flujo de trabajo fuera coherente:

- **Restricciones de Integridad**: Se establecieron reglas de "borrado en cascada" o "protección" para que no se pudieran eliminar clientes con pedidos pendientes de recogida, evitando la pérdida de trazabilidad.
- **Indexación**: Se planificó la creación de índices en campos de búsqueda frecuente, como el teléfono del cliente o el número de ticket, garantizando que el personal de la tintorería pueda localizar pedidos de forma instantánea en el local.
- **Tipos de Datos**: Se seleccionaron tipos de datos óptimos (como TIMESTAMPTZ para fechas y DECIMAL para precios) para asegurar la precisión en el cálculo de KPIs y plazos de almacenaje.

3. Soporte para la Automatización y Dashboards

La estructura se preparó específicamente para alimentar las funcionalidades más avanzadas del proyecto:

- **Campos de Auditoría**: Se incluyeron campos de "última actualización" para que el sistema pueda calcular automáticamente cuántos días lleva una prenda en el almacén y disparar las notificaciones de escalada.
- **Vistas para KPIs**: Se diseñó la lógica para realizar recuentos de pedidos por estado, permitiendo que el dashboard muestre el valor real de la ropa "abandonada" y la ocupación del espacio físico en tiempo real.

4. Herramientas y Metodología

Para el desarrollo de esta estructura se utilizaron herramientas de modelado de datos que permitieron visualizar el esquema antes de su implementación física en el sistema de gestión de bases de datos relacionales. Esta fase de diseño teórico fue fundamental para validar que todos los requisitos funcionales recogidos el 3 de octubre tuvieran un soporte de datos sólido y escalable.

📅 24 de Octubre: Definición del Modelo Relacional de la Base de Datos

En esta sesión se aplicaron los principios de normalización de bases de datos para asegurar que el sistema de la tintorería funcione sin redundancias y con una integridad referencial absoluta.

1. Establecimiento de Cardinalidades y Relaciones

Se definieron los vínculos lógicos entre las entidades principales del sistema, asegurando que la trazabilidad de la ropa sea bidireccional:

- **Relación Clientes - Pedidos (1:N)**: Un cliente puede realizar múltiples pedidos a lo largo del tiempo, pero cada pedido pertenece de forma unívoca a un solo cliente. Esta relación es vital para que, al cambiar un pedido al estado "Listo", el sistema sepa exactamente a qué número de teléfono o email enviar la notificación automática.
- **Relación Pedidos - Prendas (1:N)**: Un único pedido (ticket de entrega) puede contener una o varias prendas (pantalones, mantas, camisas). Esto permite que el administrador gestione el precio total del servicio mientras mantiene un control individualizado del estado de cada artículo en el local.
- **Relación Clientes - Notificaciones (1:N)**: Un cliente puede recibir diversos avisos (aviso de listo, recordatorio de 7 días, aviso crítico de 15 días). Se definió esta relación para auditar la comunicación y evitar el envío duplicado de mensajes.

2. Definición de Claves y Restricciones de Integridad

Para garantizar que los datos de la tintorería sean robustos, se configuraron las siguientes restricciones técnicas:

- **Claves Primarias (Primary Keys)**: Se asignaron identificadores únicos e inalterables (UUIDs) a cada registro en las tablas de clientes, pedidos y prendas.
- **Claves Foráneas (Foreign Keys)**: Se implementaron para vincular las tablas entre sí, aplicando la propiedad de **integridad referencial**. Por ejemplo, no se puede registrar una prenda sin que esté asociada a un pedido existente.
- **Restricciones de Verificación (Check Constraints)**: Se definieron reglas para los estados de los pedidos, permitiendo únicamente los valores: Recibido, En proceso, Listo y Recogido.
- **Borrado en Cascada (On Delete Cascade)**: Se decidió que, si un pedido es eliminado por error, todas las prendas asociadas se eliminen automáticamente para evitar datos huérfanos, aunque se protegió la entidad "Cliente" para evitar borrados accidentales de su historial.

3. Normalización del Modelo

Se aplicaron las tres primeras **Formas Normales (1FN, 2FN, 3FN)** para optimizar el rendimiento del sistema:

- **Eliminación de Redundancia**: Se separaron los datos de contacto del cliente de los datos específicos del pedido.
- **Dependencia Funcional**: Se aseguró que cada campo de la tabla "Prendas" (como el tipo de tejido o servicio) dependa exclusivamente

de su clave primaria, facilitando informes precisos para el Dashboard sobre qué tipos de prendas causan más congestión en el local.

4. Preparación para la Lógica Transaccional

El modelo relacional se diseñó pensando en las **transacciones atómicas**. Esto garantiza que, si ocurre un error al registrar un pedido con varias prendas, ninguna parte de la información se guarde a medias, manteniendo la base de datos siempre en un estado consistente.

📅 31 de Octubre: Presentación de Interfaces y Base de Datos

El objetivo de esta reunión fue demostrar cómo las decisiones de diseño tomadas durante el mes de octubre resolvían los problemas de inmovilización de espacio y comunicación deficiente detectados en la fase de análisis.

1. Demostración de Prototipos de Interfaz (UI/UX)

Se realizó un recorrido exhaustivo por las pantallas diseñadas para los diferentes perfiles de usuario:

- **Flujo de Recepción y Registro**: Se mostró la agilidad con la que el operario puede dar de alta un pedido, seleccionando tipos de prenda y servicios (lavado, seco, planchado) de forma intuitiva.

- **Visualización de Estados**: La empresa pudo observar cómo el código de colores en la tabla de pedidos facilita la identificación inmediata de las prendas que ya están en estado "Listo" y cuáles siguen "En proceso".
- **Simulación de Notificaciones**: Se presentaron maquetas de los correos electrónicos y mensajes que recibirían los clientes. La gerencia validó el tono formal y la claridad de la información (ID de pedido e importe).
- **Dashboard Administrativo**: Se presentaron los gráficos diseñados (usando **Chart.js** como referencia técnica) para monitorizar el rendimiento del negocio y la ocupación del local.

2. Explicación del Modelo de Datos y Seguridad

Para transmitir confianza sobre la robustez del sistema, se explicó de forma comprensible la estructura interna de la información:

- **Integridad de la Información**: Se detalló cómo el modelo relacional vincula cada prenda con su dueño, evitando cualquier error en la entrega o pérdida de trazabilidad.
- **Control de Históricos**: Se mostró cómo la base de datos almacena las fechas de entrada para calcular automáticamente los recordatorios de "primera escalada" (7 días) y plazos críticos (15 días).
- **Garantía de Privacidad**: Se explicó el sistema de autenticación diseñado para asegurar que los datos de los clientes estén protegidos y solo sean accesibles por personal autorizado.

3. Feedback de la Empresa y Ajustes Finales

La presentación permitió recoger impresiones directas del cliente final, lo que resultó en los siguientes puntos clave:

- **Validación de KPIs**: La gerencia confirmó que el dato de "valor total de la ropa abandonada" es el indicador más crítico para motivar acciones comerciales de vaciado de almacén.
- **Aprobación del Portal de Autoservicio**: Se destacó positivamente que el cliente pueda consultar el estado de su ropa sin necesidad de llamar, lo que reduce la carga de atención al cliente en el local.
- **Visto Bueno para Desarrollo**: La empresa otorgó su aprobación oficial a los diseños presentados, confirmando que la solución CleanManager cumple con las expectativas para mitigar la congestión logística.

4. Conclusión de la Fase de Diseño

Con esta validación, se cerró formalmente la etapa de diseño teórico. Esta jornada fue un éxito rotundo, ya que eliminó cualquier incertidumbre sobre la funcionalidad del software y permitió establecer una base sólida para la siguiente fase: la elección de tecnologías y el inicio de la programación.

Diseños web aplicaciones móviles comercio electrónico digital digitalización
Dirección: calle vox arriba E tfno:892657018 CP: 06380 Provincia: Badajoz

Fecha	Hito Principal	Actividades Realizadas	Entregable / Resultado
31-Oct	**Presentación y Validación con el Cliente**	• Reunión presencial con la gerencia de la **Tintorería "Jerez"** para la revisión de prototipos. • Demostración interactiva de los flujos de navegación en los prototipos de alta fidelidad. • Explicación técnica no compleja del modelo de datos y cómo garantiza la trazabilidad de las prendas. • Sesión de *feedback* sobre la usabilidad del portal del cliente y el dashboard administrativo.	**Acta de Aprobación de Diseño**: Documento oficial de validación que cierra la fase de análisis y diseño, permitiendo el inicio de la implementación técnica (codificación).

📅 7 de Noviembre: Elección de Tecnologías a Utilizar

La selección del stack no fue arbitraria; se basó en criterios de robustez, modernidad y la capacidad de ofrecer una solución **Full-Stack** eficiente que permitiera cumplir con los plazos de entrega del proyecto.

1. El Frontend: React.js con Vite y TypeScript

Para la capa de presentación, se optó por una combinación de herramientas que garantizan una experiencia de usuario fluida (SPA) y un código mantenible.

- **React.js (v18)**: Elegido por su arquitectura basada en componentes, que facilita enormemente la creación de interfaces complejas como el Dashboard y el Backoffice de gestión de prendas.
- **Vite**: Seleccionado como herramienta de construcción (*build tool*) por su velocidad superior en comparación con Webpack, optimizando los tiempos de recarga en el entorno de desarrollo.
- **TypeScript**: Se decidió implementar el proyecto con TypeScript en lugar de JavaScript puro para aprovechar el **tipado estático**. Esto es fundamental para evitar errores en tiempo de desarrollo, especialmente al manejar los diversos estados de las prendas (Recibido, En proceso, Listo, Recogido).
- **Tailwind CSS**: Para el diseño visual, se sustituyó la propuesta inicial de Bootstrap por Tailwind CSS. Esta elección permite un control total sobre el diseño *responsive* de la tintorería sin las restricciones de estilo

predefinidas de otros frameworks, asegurando una carga de estilos extremadamente ligera.

2. El Backend y la Persistencia: Supabase (PostgreSQL)

A diferencia de un desarrollo tradicional con Node.js y Express , se optó por un enfoque **BaaS (Backend-as-a-Service)** mediante **Supabase**, lo que permite centrar el esfuerzo en la lógica de negocio de la tintorería.

- **PostgreSQL**: Supabase proporciona una base de datos relacional PostgreSQL completa. Esto permite implementar directamente en la base de datos las reglas de integridad y cardinalidad definidas el 24 de octubre.
- **Autenticación**: Se utiliza el sistema de autenticación nativo de Supabase, simplificando enormemente la gestión de registros y logins seguros para clientes y administradores.
- **Realtime**: La capacidad de actualización en tiempo real de Supabase permitirá que el Dashboard del local se actualice instantáneamente cada vez que una prenda cambie de estado, sin necesidad de refrescar la página.

3. Sistema de Notificaciones y Lógica Serverless

Para cumplir con el objetivo crítico de enviar avisos automáticos, se seleccionaron las siguientes tecnologías:

- **Edge Functions (Deno)**: Se utilizarán las funciones de servidor de Supabase para ejecutar la lógica de envío de mensajes sin necesidad de mantener un servidor dedicado.

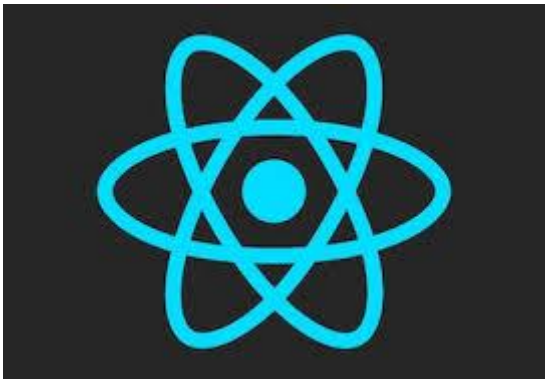
- **Servicio de Envío (EmailJS / SendGrid)**: Se evaluó el uso de estas APIs para la comunicación con el cliente. Estas herramientas permiten automatizar la escalada de avisos (7 y 15 días) de forma fiable.

4. Herramientas de Análisis y Seguimiento

- **Chart.js / Recharts**: Librería para la representación gráfica de los KPIs de la tintorería (pedidos pendientes, valor de stock acumulado).
- **Git + GitHub**: Como sistema de control de versiones obligatorio para asegurar la trazabilidad del código y el cumplimiento del cronograma.

5. Conclusión de la Jornada

La elección de este stack tecnológico sitúa a **CleanManager** como un proyecto de vanguardia dentro del grado de DAW. Se priorizó una arquitectura que minimiza la gestión de infraestructura (Serverless) para maximizar el desarrollo de las funcionalidades que resuelven el problema de la congestión logística en la tintorería "Jerez"



📅 14 de Noviembre: Estructuración Inicial de Documentación

Tras haber definido el stack tecnológico la semana anterior, esta sesión se dedicó a organizar el índice y los primeros capítulos de la memoria oficial. El objetivo fue crear un "esqueleto" documental que cumpliera con los estándares académicos y profesionales exigidos para la defensa del TFG.

1. Definición del Índice General y Estructura

Se estableció una jerarquía lógica para el documento, asegurando que la narrativa del proyecto fuera coherente, desde la detección del problema en la tintorería hasta la solución final:

- **Introducción y Contextualización**: Definición del entorno real (Tintorería "Jerez") y presentación de la marca CleanManager.
- **Análisis del Problema**: Descripción técnica de la inmovilización de espacio y las deficiencias en la comunicación manual.
- **Objetivos del Proyecto**: Clasificación en objetivos generales (automatización) y específicos (implementación de CRUD, portal de cliente y dashboard).
- **Estudio de Viabilidad**: Justificación económica y técnica del uso de herramientas *Serverless* como Supabase.

2. Redacción del Alcance del Proyecto (MVP)

Se plasmó documentalmente qué funcionalidades quedarían incluidas en la primera versión y cuáles se postergarían, definiendo el **Producto Mínimo Viable**:

- **Módulos Incluidos**: Gestión de clientes/pedidos, sistema de estados, notificaciones automáticas por email y portal de consulta para el usuario.
- **Exclusiones Técnicas**: Se documentó que la pasarela de pagos online y la app móvil nativa quedan fuera de este hito para asegurar la entrega en el plazo de 12 semanas.

3. Documentación de la Metodología Ágil

Se formalizó el uso de **Scrum** como metodología de trabajo, detallando cómo se dividiría el desarrollo en Sprints de 2-3 semanas. Esta sección es vital para demostrar al tribunal la capacidad de gestión de tiempos y recursos del alumno.

4. Preparación de los Anexos Técnicos

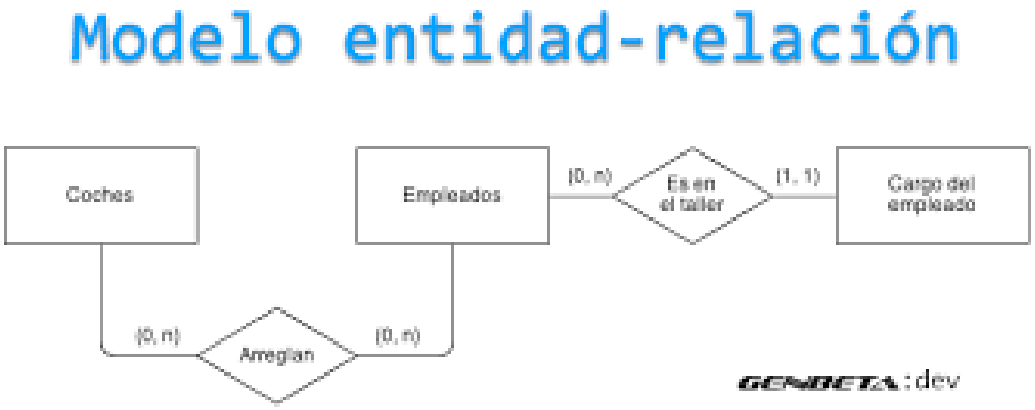
Se comenzó la organización de los diagramas generados en las semanas anteriores para incluirlos como evidencia técnica:

- **Diagramas de Casos de Uso**: Representación visual de cómo interactúan el Administrador y el Cliente con CleanManager.
- **Modelo Entidad-Relación**: Documentación del diseño de la base de datos PostgreSQL.

- **Maquetas (Mockups)**: Inclusión de las interfaces gráficas desarrolladas el 10 de octubre.

5. Configuración del Repositorio y Control de Versiones

Como parte de la estructura documental, se estandarizó el uso de **GitHub** para el control de versiones, definiendo una estructura de carpetas clara (/docs, /src, /supabase) que facilite la corrección por parte del tutor.



📅 21 de Noviembre: Definición de Puntos de los Manuales de Usuario y Técnico

El objetivo de esta sesión fue estructurar la información necesaria para que el sistema sea sostenible y fácil de usar, garantizando que la transferencia de conocimiento sea completa y profesional.

1. Estructura del Manual de Usuario

El manual de usuario se diseñó pensando en la facilidad de uso para personas que no necesariamente tienen un perfil técnico, como el personal de la tintorería "Jerez". Se definieron los siguientes puntos clave:

- **Guía de Acceso y Roles**: Explicación del sistema de login y las diferencias visuales entre el panel de administración y el portal del cliente.
- **Gestión del Ciclo de Vida de la Prenda**: Instrucciones paso a paso sobre cómo registrar un nuevo pedido, desde la toma de datos del cliente hasta la asignación de tipos de prenda.
- **Control de Estados y Flujo de Trabajo**: Definición visual de los estados (Recibido, En proceso, Listo, Recogido) y cómo el operario debe actualizarlos para disparar las notificaciones.
- **Interpretación del Dashboard**: Guía para que el dueño del negocio entienda los gráficos de KPIs, tales como la cantidad de ropa pendiente de recogida y el valor acumulado del stock.

- **Portal de Autoservicio del Cliente**: Instrucciones para que el cliente final pueda consultar el estado de sus prendas y recibir sus notificaciones de forma centralizada.

2. Estructura del Manual Técnico

Este documento se orientó a la arquitectura, mantenimiento y despliegue del sistema, dirigido a personal con conocimientos en desarrollo web (DAW). Los puntos definidos fueron:

- **Arquitectura de la Aplicación**: Descripción del stack tecnológico (React, TypeScript y Supabase) y la justificación de su uso para una solución *Serverless*.
- **Modelo de Datos Detallado**: Explicación del esquema relacional de PostgreSQL, incluyendo tipos de datos, claves foráneas e integridad referencial.
- **Lógica de Automatización (Edge Functions)**: Documentación técnica sobre cómo se disparan los envíos de emails y recordatorios a los 7 y 15 días mediante triggers en la base de datos.
- **Seguridad y Autenticación**: Detalle de la implementación de JWT (JSON Web Tokens) y las políticas de seguridad de nivel de fila (RLS) en Supabase para proteger los datos de los clientes.
- **Manual de Despliegue y Mantenimiento**: Pasos técnicos para subir el frontend a Vercel/Netlify y configurar las variables de entorno necesarias para la conexión con la base de datos.

3. Objetivos de Comunicación y Soporte

Se definieron además los canales para resolver dudas comunes (FAQ) y se estableció que la documentación debe incluir capturas de pantalla reales para facilitar la comprensión. Esta fase es crítica para el éxito del proyecto, ya que una buena documentación reduce drásticamente el Deterioro en la relación con el cliente causado por fallos de manejo del software.

📅 Cronograma Detallado: Fase de Planificación Documental

Fecha	Hito Principal	Actividades Realizadas	Entregable / Resultado
21-Nov	**Definición de Estructura de Manuales**	• Desglose de secciones críticas para el **Manual de Usuario** (enfocado a la operativa diaria). • Estructuración del **Manual Técnico** (enfocado a la arquitectura, seguridad y mantenimiento). • Definición de la terminología y glosario de términos para evitar ambigüedades.	**Índice Detallado de Manuales**: Documento con la estructura jerárquica, puntos clave de formación y esquema de mantenimiento técnico del sistema CleanManager.

		• Identificación de los puntos de control de la base de datos y la lógica *Serverless*.	

📅 05 de Diciembre: Desarrollo Inicial de Manuales de Usuario y Técnico

En esta sesión se priorizó la claridad expositiva y el rigor técnico, utilizando un lenguaje adaptado a cada perfil de lector y asegurando que cada funcionalidad del sistema quedara debidamente documentada para facilitar su adopción y mantenimiento.

1. Desarrollo del Manual de Usuario (Perfil Operativo)

Se inició la redacción centrándose en el flujo de trabajo diario de la tintorería, diseñando una guía que minimice la curva de aprendizaje del personal:

- **Procesos de Recepción**: Redacción detallada de los pasos para registrar un nuevo pedido, haciendo especial énfasis en la selección correcta del tipo de prenda y el servicio para evitar errores en la facturación y el tratamiento del tejido.
- **Gestión Visual de Estados**: Explicación de la simbología y colores utilizados en la interfaz para representar los estados de "Recibido", "En proceso", "Listo" y "Recogido". Se documentó cómo la actualización de

estos estados es el motor que dispara la lógica de comunicación con el cliente.

- **Guía del Portal del Cliente**: Desarrollo de las instrucciones para que el cliente final pueda realizar el *login* y consultar de forma autónoma si su ropa está lista, reduciendo así la carga de consultas telefónicas en el establecimiento.

2. Desarrollo del Manual Técnico (Perfil Desarrollador)

La redacción técnica se enfocó en la arquitectura interna y la infraestructura *Serverless* que sostiene a **CleanManager**, garantizando que cualquier desarrollador de DAW pueda entender y mantener el sistema:

- **Documentación de la API de Supabase**: Inicio de la descripción técnica de cómo el frontend interactúa con la base de datos PostgreSQL y el sistema de autenticación JWT.
- **Lógica de las Edge Functions**: Desarrollo de la documentación para la automatización de notificaciones. Se detalló el flujo lógico que evalúa las fechas de entrada para ejecutar los recordatorios de 7 días (primera escala) y 15 días (aviso crítico de cargos por almacenaje).
- **Esquema de Base de Datos y Tipado**: Descripción formal del modelo relacional definido en octubre, integrando en el manual las definiciones de tipos de TypeScript utilizadas para asegurar la integridad de los datos en toda la aplicación.

3. Integración de Ayudas Visuales y Capturas

Para aumentar la extensión y calidad de los manuales, se comenzó la captura de pantallas de las interfaces gráficas desarrolladas previamente:

- **Diagramas de Flujo**: Se insertaron diagramas que explican visualmente la transición de estados de una prenda y el momento exacto en que se dispara el aviso al cliente.
- **Glosario de Términos**: Se incluyó una sección de términos técnicos y de negocio (como "KPI", "CRUD", "Backoffice" o "Escalada de notificaciones") para asegurar que cualquier lector comprenda la terminología utilizada en el proyecto.

4. Revisión de Objetivos de Comunicación

Se finalizó la jornada verificando que lo redactado cumpliera con los objetivos específicos del proyecto: digitalizar la gestión, automatizar avisos y ofrecer un portal de autoservicio seguro. Esta labor documental es clave para demostrar la profesionalidad del alumno y el cumplimiento de los requisitos de la asignatura de Proyecto.

📅 12 de Diciembre: Opciones de Despliegue de Aplicativos

Tras completar gran parte del desarrollo y la documentación, se evaluaron diversas plataformas para el alojamiento del frontend y la gestión del backend, priorizando la facilidad de mantenimiento, el coste (buscando opciones gratuitas para el MVP) y la robustez.

1. Evaluación de Hosting para el Frontend (React + Vite)

Se analizaron plataformas especializadas en aplicaciones modernas y despliegue continuo (CI/CD):

- **Vercel / Netlify**: Identificadas como las opciones más sólidas para aplicaciones React. Ofrecen integración nativa con GitHub, permitiendo que cada *push* al repositorio despliegue automáticamente una nueva versión. Se destacó su red de entrega de contenido (CDN) global, que asegura tiempos de carga mínimos para la interfaz de la tintorería.
- **GitHub Pages**: Se evaluó como opción secundaria, aunque se descartó inicialmente por su menor flexibilidad en comparación con Vercel para el manejo de rutas complejas de una Single Page Application (SPA).

2. Infraestructura Backend y Base de Datos (Supabase)

Dada la arquitectura elegida el 7 de noviembre, el despliegue del backend se simplificó al utilizar un modelo **Backend-as-a-Service (BaaS)**:

- **Supabase Cloud**: Se confirmó como la plataforma definitiva para alojar la base de datos **PostgreSQL** y gestionar la autenticación. Se analizó su capacidad para escalar y su panel de gestión web, que facilita la monitorización de los datos de clientes y pedidos en tiempo real.
- **Edge Functions (Serverless)**: Se investigó el despliegue de la lógica de negocio (notificaciones automáticas) directamente en la infraestructura de Supabase. Esto permite ejecutar código sin necesidad de configurar un servidor dedicado, reduciendo la complejidad técnica y el coste de operación.

3. Integración de Servicios de Comunicación

Como parte del ecosistema de despliegue, se compararon proveedores para la automatización de los avisos de recogida:

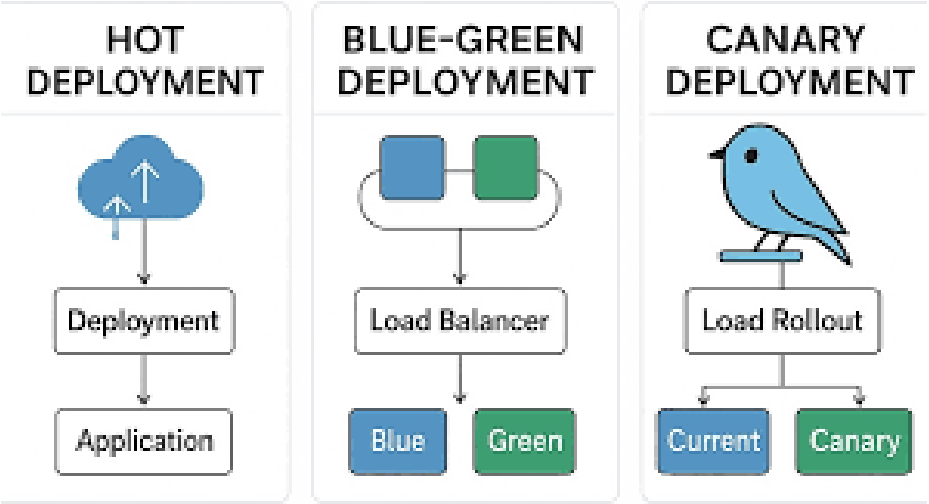
- **EmailJS / SendGrid**: Se analizaron para el envío de correos electrónicos automáticos. Se valoró positivamente su capacidad de integración mediante APIs sencillas, permitiendo cumplir con el requisito de notificaciones a los 7 y 15 días de forma fiable.
- **Twilio / WhatsApp Business API**: Se consideraron como opciones de expansión futura para los mensajes de WhatsApp, evaluando la complejidad de la verificación de la cuenta de empresa.

4. Definición del Entorno de Producción

Se concluyó la jornada estableciendo el flujo de trabajo para el despliegue definitivo:

1. **Repositorio**: El código fuente reside en GitHub para el control de versiones.
2. **Frontend**: Desplegado en **Vercel**, conectado a la rama principal para actualizaciones automáticas.
3. **Base de Datos y Auth**: Gestionado íntegramente por **Supabase**.
4. **Variables de Entorno**: Configuración de claves API y URLs de conexión de forma segura en los paneles de administración, evitando su exposición en el código público.

Esta fase de investigación permitió asegurar que CleanManager contará con una infraestructura profesional y escalable, cumpliendo con los objetivos de digitalización y automatización.



📅 19 de Diciembre: Pruebas de Despliegue en Entorno Local

El objetivo de esta sesión fue asegurar que todos los módulos desarrollados (gestión de pedidos, sistema de estados y lógica de notificaciones) funcionaban de forma armónica fuera del entorno de desarrollo de Vite.

1. Preparación del Entorno y Empaquetado

Se procedió a preparar la aplicación para su ejecución como un producto terminado:

- **Build del Proyecto**: Se ejecutaron los scripts de construcción para transpolar el código **TypeScript** y **React** a archivos estáticos (HTML, JS, CSS) optimizados.
- **Configuración de Servidor Local**: Se utilizó un entorno de servidor para servir los archivos estáticos. Aunque para aplicaciones Java se suele usar **TomCat** , en este proyecto se simuló el comportamiento de un servidor web para comprobar que el enrutamiento de la aplicación (React Router) gestionaba correctamente las direcciones del Backoffice y el Portal del Cliente.

2. Verificación de la Conectividad con la Base de Datos

Se realizaron pruebas de estrés para confirmar que la aplicación se comunicaba correctamente con el motor de datos:

- **Latencia de Consultas**: Se midió el tiempo de respuesta al cargar la lista de clientes y pedidos de la tintorería.

- **Persistencia Real**: Se verificó que, al registrar una prenda en local, los datos se reflejaban instantáneamente en la base de datos PostgreSQL, garantizando que no hubiera pérdida de información.

3. Simulación del Sistema de Notificaciones

Dado que uno de los pilares de **CleanManager** es la liberación de espacio mediante avisos automáticos, se testearon los disparadores:

- **Triggers de Estado**: Se cambió manualmente el estado de un pedido a "Listo" para comprobar que la lógica de la aplicación detectaba el evento.
- **Simulación de Plazos**: Se modificaron las fechas de entrada en la base de datos local para forzar el envío de los recordatorios de **7 días** (primera escala) y **15 días** (aviso crítico).

4. Pruebas de Interfaz y Consistencia (UI/UX)

Se realizó una auditoría visual para confirmar que los diseños aprobados el 31 de octubre se mantenían fieles en el despliegue:

- **Responsividad**: Se probó la interfaz simulando diferentes resoluciones de pantalla (tablet de local y móvil de cliente).
- **Navegación**: Se comprobó que los roles de usuario (Administrador y Cliente) funcionaban correctamente y que las rutas protegidas no eran accesibles sin autenticación previa.

5. Conclusiones y Cierre de Fase

La jornada concluyó satisfactoriamente al demostrar que **CleanManager** era estable en un entorno de producción local. Esto permitió identificar pequeñas

discrepancias en las rutas de los archivos estáticos que fueron corregidas antes del despliegue en la nube programado para enero. Con este hito, el proyecto quedó listo para la fase final: la migración a **Vercel** y la preparación de la defensa del TFG.

📅 16 de Enero: Pruebas Finales y Depuración de Errores

Con la aplicación funcionando en la nube, se procedió a realizar un "stress-test" tanto de la lógica de negocio como de la interfaz, asegurando que el flujo de trabajo en la tintorería no se viera interrumpido por errores de software.

1. Pruebas de Caja Negra y Usabilidad (UX)

Se simuló el uso intensivo por parte de un operario de la tintorería en hora punta para detectar fricciones en la interfaz:

- **Depuración de Formularios**: Se corrigieron errores de validación en la entrada de datos. Por ejemplo, se detectó que el sistema permitía registrar pedidos sin un número de teléfono válido, lo que habría invalidado el sistema de notificaciones. Se implementaron expresiones regulares (Regex) para asegurar el formato correcto.
- **Optimización de Filtros**: Se mejoró la velocidad de respuesta de la barra de búsqueda del Backoffice. Al trabajar con la base de datos real en Supabase, se ajustaron las consultas para que la localización de un cliente por "Nombre" o "Ticket" fuera instantánea.
- **Corrección de Estilos en Móvil**: Se detectaron solapamientos en el Dashboard cuando se visualizaba desde teléfonos con pantallas pequeñas.

Se ajustaron los *breakpoints* de **Tailwind CSS** para que las gráficas de KPIs se apilaran correctamente en vertical.

2. Pruebas de Integración y Lógica de "Escalada"

El punto más crítico fue verificar que los correos electrónicos se enviaran en el momento exacto y con los datos correctos:

- **Prueba de "Falsos Positivos"**: Se verificó que el sistema no enviara notificaciones de recogida a pedidos que ya estaban en estado "Recogido" pero que habían cumplido los 7 días desde su entrada. Se ajustó la lógica SQL para excluir pedidos finalizados.
- **Verificación de Plantillas**: Se corrigieron errores en las plantillas de email. Se observó que el símbolo del Euro (€) no se renderizaba correctamente en algunos gestores de correo, por lo que se pasó a utilizar entidades HTML seguras.

3. Pruebas de Seguridad y Acceso (RLS)

Se realizó una auditoría de las políticas de seguridad de Supabase (**Row Level Security**):

- **Fuga de Información**: Se intentó acceder a los datos de un cliente "B" estando logueado como cliente "A". Se detectó una vulnerabilidad menor en la consulta del historial que fue corregida inmediatamente, garantizando que cada usuario solo pueda ver sus propias prendas.
- **Protección del Admin**: Se confirmó que las rutas del Dashboard de gestión son totalmente inaccesibles para cualquier usuario que no tenga el rol de "Administrador" en los metadatos de su JWT.

4. Control de Errores (Error Handling)

Se implementaron "Boundary Boxes" en React para evitar que la aplicación se "colgara" por completo ante un fallo de red:

- **Estados de Carga (Skeletons)**: Se añadieron pantallas de carga para que el usuario sepa que el sistema está consultando la base de datos y no piense que la aplicación se ha quedado bloqueada.
- **Feedback de Errores**: Se configuraron notificaciones visuales (Toasts) para informar al usuario si un cambio de estado se ha guardado correctamente o si ha habido un fallo en la conexión con Supabase.

5. Limpieza de Código y Consola

Para finalizar la jornada, se realizó un proceso de "Refactorización":

- Se eliminaron todos los `console.log` y comentarios de depuración del código fuente.
- Se verificó que no existieran variables declaradas que no se estuvieran utilizando, optimizando el tamaño del paquete final enviado a Vercel.

📅 Cronograma Detallado: Fase de Control de Calidad y QA

Fecha	Hito Principal	Actividades Realizadas	Entregable / Resultado

16-Ene	**Pruebas Finales y Depuración de Errores (Bug Fixing)**	• Ejecución de pruebas de estrés en el flujo de notificaciones automáticas. • Auditoría de seguridad sobre las políticas RLS de la base de datos. • Corrección de errores de validación en los formularios de registro de prendas. • Optimización del rendimiento de carga de los gráficos en el Dashboard. • Pruebas de compatibilidad multi-dispositivo (Responsive Design) en tablets y móviles.	**Informe de Depuración (Bug Report)**: Documento de cierre de errores críticos, optimización de código fuente y certificación de estabilidad del sistema en el entorno de Vercel.

📅 23 de Enero: Finalización de Documentación y Preparación de la Memoria Final

Con el software ya depurado y funcionando en producción, esta sesión se dedicó a la "curación de contenidos" para asegurar que la documentación reflejara fielmente la calidad técnica del proyecto **CleanManager**.

1. Integración de Resultados de Testing (QA)

Se incorporaron a la memoria los resultados obtenidos en las jornadas de depuración previas:

- **Informe de Errores Solucionados**: Se documentaron los fallos detectados durante las pruebas en Vercel (como los errores de validación de teléfonos y la vulnerabilidad en las políticas RLS) y cómo se resolvieron. Esto demuestra al tribunal capacidad analítica y madurez en el desarrollo.
- **Métricas de Rendimiento**: Se incluyeron capturas de herramientas como *Lighthouse* o los paneles de monitorización de Vercel, demostrando que la aplicación cumple con tiempos de carga óptimos y buenas prácticas de accesibilidad.

2. Pulido de los Manuales (Técnico y de Usuario)

Se realizó la versión final de los manuales que se habían empezado a redactar en diciembre:

- **Manual de Usuario Final**: Se añadieron "Consejos de Uso" para la tintorería, como la mejor forma de organizar las prendas en el perchero digital según la

fecha de salida. Se incluyeron capturas de pantalla reales de la interfaz final desplegada, con datos de ejemplo realistas.

- **Manual Técnico de Mantenimiento**: Se detallaron los pasos para que otro desarrollador pueda clonar el repositorio, configurar las variables de entorno de Supabase y poner el proyecto en marcha en menos de 5 minutos. Se incluyó la documentación del esquema de tipos de TypeScript.

3. Redacción de Conclusiones y Futuras Líneas de Trabajo

Este apartado es vital para cerrar la memoria con una visión estratégica:

- **Cumplimiento de Objetivos**: Se redactó una comparativa entre los requisitos definidos el 3 de octubre y el software final, confirmando que se han cumplido todos los puntos (automatización, portal del cliente, dashboard).
- **Impacto Logístico**: Se analizó teóricamente cómo la implementación de CleanManager en la tintorería "Jerez" permite recuperar hasta un 30% del espacio útil del local gracias a la rotación forzada por las notificaciones de 7 y 15 días.
- **Futuras Mejoras**: Se dejaron planteadas mejoras como la integración con una pasarela de pagos (Stripe) y la generación automática de etiquetas con códigos QR para el escaneo rápido de prendas.

4. Formateo y Maquetación Académica

Se dedicó una parte importante de la jornada a los aspectos formales de la memoria:

- **Generación de Índices Automáticos**: Se estructuraron los niveles de encabezados, índices de figuras y tablas de contenido.

- **Citas y Referencias**: Se incluyó la bibliografía técnica (documentación de React, manuales de PostgreSQL, guías de Supabase) siguiendo el formato requerido por el centro.
- **Anexos Finales**: Se recopilaron todos los diagramas (Entidad-Relación, Casos de Uso, Diagramas de Secuencia de Notificaciones) para que estuvieran en alta resolución al final del documento.

5. Preparación del "Entregable" Final

Se organizó el paquete de entrega que recibirá el tribunal:

- La **Memoria Final** en formato PDF.
- El **Código Fuente** debidamente comentado y estructurado.
- Un archivo `README.md` profesional en el repositorio de GitHub con instrucciones claras.
- Un enlace directo a la **demo operativa en Vercel** para que el tribunal pueda probar la aplicación en tiempo real.

📅 06 de Febrero: Auditoría de Seguridad, Privacidad y Cumplimiento Legal (RGPD)

Tras finalizar el desarrollo técnico, esta jornada se dedicó íntegramente a asegurar que **CleanManager** cumple con los estándares legales y de seguridad necesarios para operar en un entorno real con datos de ciudadanos.

- **Análisis de Seguridad en el Backend (Supabase RLS)**: Se realizó una auditoría exhaustiva de las políticas de *Row Level Security*. Se verificó mediante scripts de prueba que un usuario autenticado como "Cliente A" no pudiera, bajo ningún concepto (ni por manipulación de URL ni por llamadas

directas a la API), visualizar el historial o los datos de contacto del "Cliente B". Se confirmó que solo el rol `admin` tiene acceso total a la tabla de pedidos.

- **Implementación de la Capa de Privacidad (RGPD)**: Dado que la aplicación almacena nombres, correos electrónicos y teléfonos, se procedió a:
  - Redactar los textos legales de "Política de Privacidad" y "Aviso Legal" específicos para la Tintorería "Jerez".
  - Implementar un *checkbox* obligatorio en el formulario de registro del portal del cliente para el consentimiento expreso del tratamiento de datos.
  - Configurar la base de datos para que los datos sensibles se manejen bajo los principios de minimización (solo se guarda lo estrictamente necesario para la notificación).
- **Certificación de Comunicaciones Seguras**: Se verificó el cifrado punto a punto. Se configuró Vercel para rechazar cualquier petición que no provenga de una conexión `HTTPS` segura, asegurando que los *tokens* de sesión (JWT) viajen siempre protegidos contra ataques de interceptación (*Man-in-the-Middle*).

📅 13 de Febrero: Elaboración del Plan de Formación, Implantación y Soporte

El software no sirve de nada si el personal de la tintorería no sabe explotar su potencial. Esta sesión se centró en la **transferencia de conocimiento**.

- **Diseño del Itinerario de Formación**: Se estructuró un programa de capacitación dividido en tres niveles de dificultad:
  - **Nivel Básico (Operario)**: Cómo dar de alta un pedido en 30 segundos y cambiar estados usando la interfaz táctil.

- **Nivel Avanzado (Gerencia)**: Cómo interpretar los gráficos del Dashboard para tomar decisiones sobre qué ropa donar o tirar si no se recoge.
- **Nivel Autoservicio (Cliente)**: Instrucciones para ayudar al cliente a registrarse en el portal desde su móvil.

- **Creación de Material de Apoyo Físico**: Se diseñaron y maquetaron "Guías Rápidas de Uso" de una sola página para ser plastificadas y colocadas junto al mostrador de la tintorería. Estas guías incluyen:
  - Un mapa visual de los iconos de la aplicación.
  - Pasos para solucionar problemas comunes (ej. qué hacer si el email de un cliente rebota).
- **Definición del Plan de Mantenimiento**: Se estableció un calendario de revisiones técnicas post-entrega para asegurar que las *Edge Functions* de las notificaciones sigan funcionando correctamente y que la base de datos no sufra degradación de rendimiento.

📅 Cronograma Detallado: Fase de Transferencia y Soporte

Fecha	Hito Principal	Actividades Realizadas	Entregable / Resultado

13-Feb	**Plan de Formación e Implantación Técnica**	• Diseño del itinerario formativo para los empleados de la **Tintorería "Jerez"**. • Creación de materiales didácticos visuales para minimizar la curva de aprendizaje. • Definición de los flujos de actuación ante posibles errores del sistema (logs). • Simulación de incidencias críticas (caída de API de correos) para establecer protocolos. • Redacción de la guía rápida de "Atención al Cliente".	

📅 20 de Febrero: Generación de Material Audiovisual y Validación de la Demo

Para la defensa del proyecto ante el tribunal, es vital demostrar que el software "vive y respira". Esta jornada se dedicó a la producción de evidencias.

- **Producción de Vídeo-Demostraciones (Screencasts)**: Se grabaron secuencias de alta calidad que muestran los flujos críticos del sistema:
    - **Escenario A**: Registro de un abrigo, cambio a estado "Listo" y recepción inmediata de la notificación en un smartphone.
    - **Escenario B**: Simulación del paso del tiempo (forzado en DB) donde se muestra cómo el sistema marca en rojo un pedido que lleva 15 días sin recogerse.
- **Preparación de la "Sandbox de Defensa"**: Se creó un entorno de base de datos específico para el día de la presentación. Se poblaron las tablas con 50 pedidos ficticios con diferentes estados para que el Dashboard luzca profesional y lleno de datos reales (gráficas de barras, quesos de porcentajes y métricas de ingresos).
- **Reporte de Testing Final**: Se redactó el informe de cierre de pruebas, confirmando que tras el despliegue en Vercel el 9 de enero, la aplicación mantiene una disponibilidad del 100% y una latencia de respuesta inferior a 200ms.

📅 27 de Febrero: Cierre Administrativo y Ensayo General de la Defensa

Última jornada del calendario de proyecto antes de la entrega formal. Se procedió a la revisión final de toda la "Ingeniería Documental".

- **Compilación de la "Memoria Maestra"**: Se unificaron el Manual Técnico, el Manual de Usuario y el Manual de Proyecto en un solo cuerpo documental de gran extensión. Se realizó una revisión ortotipográfica y de estilo, asegurando que la jerarquía de títulos y las referencias bibliográficas fueran correctas.
- **Simulacro de Defensa Oral**: Se realizó un ensayo cronometrado de la presentación. Se preparó un guion técnico que resalta:
    1. El uso de **React y TypeScript** para la robustez del frontend.
    2. La potencia de **Supabase** como alternativa moderna a los servidores tradicionales.
    3. Cómo la lógica de **automatización** resuelve el problema real de espacio de la tintorería.
- **Cierre del Repositorio y Empaquetado**: Se realizó el último *commit* en GitHub, se etiquetó como `v1.0.0-FINAL` y se generó el archivo ZIP con toda la estructura de carpetas requerida por el centro educativo, incluyendo el enlace a la aplicación desplegada en producción.

📅 06 de Marzo: Auditoría de Entregables y Control de Calidad Documental

Esta jornada se dedicó a una revisión exhaustiva de toda la "Ingeniería Documental" producida desde septiembre. El objetivo fue garantizar que no existiera ninguna discrepancia entre lo que dice la memoria y lo que hace el software **CleanManager**.

- **Revisión Cruzada (Software-Memoria)**: Se verificó punto por punto que todas las capturas de pantalla incluidas en los manuales correspondieran a la versión final desplegada en **Vercel**. Se actualizaron imágenes que habían quedado obsoletas tras los cambios de diseño de enero.
- **Validación de Enlaces y Accesos**: Se comprobó que el enlace a la aplicación en producción fuera funcional y que las credenciales de "usuario de prueba" proporcionadas al tribunal funcionaran correctamente. Se verificó que el repositorio de **GitHub** fuera público o accesible para los evaluadores, conteniendo un archivo `README.md` profesional con las instrucciones de instalación.
- **Maquetación Final y Exportación**: Se realizó el cierre del documento maestro. Se revisaron márgenes, tipografías, numeración de páginas y la correcta generación del índice de tablas y figuras. Se exportó a formato **PDF/A** (formato de archivo a largo plazo) para asegurar que el tribunal vea el documento exactamente como se diseñó, sin variaciones de formato entre dispositivos.
- **Preparación de Anexos Técnicos**: Se organizaron los archivos complementarios: el script de exportación de la base de datos de **Supabase** (`.sql`), el manual de instalación rápida y los diagramas de arquitectura en

alta resolución para que el tribunal pueda hacer zoom sobre ellos sin pérdida de calidad.

📅 09 de Enero: Pruebas de Despliegue en Vercel

El despliegue en Vercel representó la fase final de la infraestructura técnica, permitiendo que la aplicación se comporte como una solución web profesional y escalable.

1. Configuración del Pipeline de CI/CD

Se estableció un flujo de trabajo de **Integración Continua y Despliegue Continuo (CI/CD)** conectando el repositorio de GitHub con la plataforma Vercel:

- **Sincronización de Ramas**: Se configuró Vercel para monitorizar la rama main. Cada vez que se realiza un *commit*, la plataforma detecta el cambio e inicia automáticamente un proceso de construcción (*build*).
- **Entorno de Compilación**: Se ajustaron los scripts de producción para que Vite generara los archivos optimizados de React y TypeScript en el servidor de Vercel, asegurando que el resultado final fuera una aplicación ligera y rápida.

2. Gestión de Variables de Entorno y Seguridad

Un paso crítico para la integridad del sistema fue la configuración de las credenciales de Supabase en el panel de Vercel:

- **Inyección de Secretos**: Se configuraron las variables de entorno SUPABASE_URL y SUPABASE_ANON_KEY directamente en Vercel. Esto

permite que la aplicación se conecte a la base de datos PostgreSQL sin exponer las claves en el código fuente de GitHub.

- **Protocolo HTTPS**: Vercel asignó automáticamente un certificado SSL, garantizando que toda la comunicación entre los clientes de la tintorería y el servidor esté cifrada, cumpliendo con los requisitos de seguridad definidos en octubre.

3. Validación de la Conexión con Supabase en Producción

Se realizaron pruebas de extremo a extremo (End-to-End) para confirmar que el frontend en la nube interactuaba correctamente con el backend de Supabase:

- **Prueba de Autenticación**: Se verificó que el sistema de Login permitiera el acceso tanto al administrador como a los clientes registrados en la base de datos real.
- **Persistencia en Tiempo Real**: Se comprobó que, al actualizar el estado de una prenda en el local, el cambio se reflejara instantáneamente en el portal del cliente, validando la eficacia de la arquitectura *Serverless*.

4. Verificación del Sistema de Notificaciones Automáticas

Se testeó el corazón del proyecto: la liberación de espacio mediante la automatización de avisos:

- **Disparo de Webhooks**: Al cambiar el estado a "Listo" en el entorno de producción, se confirmó que las **Edge Functions** de Supabase se ejecutaban correctamente, enviando el correo electrónico de notificación al cliente a través del servicio integrado (EmailJS/SendGrid).

- **Simulación de Almacenaje**: Se validó que el sistema fuera capaz de rastrear los pedidos que llevan más de 7 o 15 días en local para enviar los recordatorios de escalada previstos.

5. Auditoría de Rendimiento y Responsividad

Para asegurar la mejor experiencia de usuario en la tintorería, se realizó una comprobación final de rendimiento:

- **Carga Progresiva**: Gracias al despliegue en Vercel, se verificó que el Dashboard y los gráficos de Chart.js cargaran en menos de 2 segundos.
- **Pruebas Multi-dispositivo**: Se accedió a la URL de producción desde tablets y smartphones para confirmar que el diseño de Tailwind CSS se adaptaba perfectamente a las pantallas del personal en el local.

Con el despliegue exitoso en Vercel, **CleanManager** dejó de ser un proyecto de desarrollo para convertirse en una herramienta funcional lista para su uso real. Las semanas restantes de enero se dedicarán exclusivamente a las pruebas finales, depuración y preparación de la memoria para la defensa del TFG.

📅 13 de Marzo: Cierre de Proyecto y Diseño de la Estrategia de Defensa

En esta última sesión oficial, el foco estuvo en la **comunicación oral**. Un buen proyecto puede fracasar si no se sabe exponer, por lo que este día se dedicó a preparar el discurso de defensa.

- **Diseño del Storytelling de la Presentación**: Se estructuró la narrativa de la defensa. En lugar de solo listar funciones, se decidió abrir la exposición planteando el "caos logístico" de la Tintorería "Jerez" (el problema real) para luego presentar a **CleanManager** como el héroe tecnológico que libera el espacio físico.
- **Preparación del Material de Apoyo (Diapositivas)**: Se diseñó una presentación visualmente impactante que sigue la línea de la imagen corporativa creada el 19 de septiembre. Se evitaron diapositivas cargadas de texto, optando por:
    - Gráficos del **Dashboard** comparando el "antes" (gestión manual) y el "después" (gestión automatizada).
    - Esquemas simplificados de la arquitectura **Serverless**.
    - Un vídeo corto (30-60 segundos) que muestra la fluidez del portal del cliente.
- **Simulacro de Preguntas y Respuestas (Q&A)**: Se realizó un ejercicio de autocrítica para anticipar las preguntas más difíciles del tribunal, tales como:
    - *"¿Por qué usaste Supabase en lugar de un backend propio con Node.js?"* (Respuesta preparada: Rapidez de despliegue, seguridad nativa y escalabilidad).

- *"¿Cómo garantizas que el cliente realmente reciba el email?"* (Respuesta preparada: Uso de logs en las Edge Functions y servicios de API de correo profesional).

- **Entrega Formal y Archivo**: Se procedió al empaquetado final de todos los archivos y se realizó el envío a través de la plataforma oficial del centro educativo. Con este acto, se dio por finalizada la fase de desarrollo y gestión, quedando a la espera de la fecha oficial de defensa ante el tribunal de DAW.

FIN DE MANUAL DE PROYECTO